



5.20 Concurrency control protocol

Concurrency control protocol should not allow to execute:

1. Non-serializable schedule (Violate Isolation).
2. Non-strict recoverable schedule (Violate atomicity and Durability).

5.21 Locking Protocol

Lock is as variable used to identify the status of data item.

Transaction (T_1)

Lock (A): Granted by concurrency controller

R(A)

W(A)

Lock(B): Denied by concurrency controller

5.22 Types of Lock

1. Shared Lock(s): Read only lock.
2. Exclusive lock (x): Read/write lock.

1. Shared (s mode)

- Exists when concurrent transaction granted READ access.
- Produce no conflict for Read only transactions.
- Issued when transaction wants to read and exclusive lock not held on item.

2. Exclusive (x) mode

- Exists when access reserved for locking transaction.
- Used when potential for conflict exists.
- Issued when transaction wants to updated unlocked data.

Example:

Transaction (T_1)

S(A) : Granted shared lock

R(A) } only read allowed

X(B) : Granted exclusive lock

R(B) }
W(B) } Read and write

5.22.1 Lock compatible table

Data item A		S	X	← Held by T_i
Requested by T_j	S	Yes	No	
	X	No	No	

5.23 Phase Locking Protocol (2PL)

- It guaranteed serializability.
- Transaction (T) allowed to request lock on any data item in any mode (x/s) until first unlock of transaction (T).

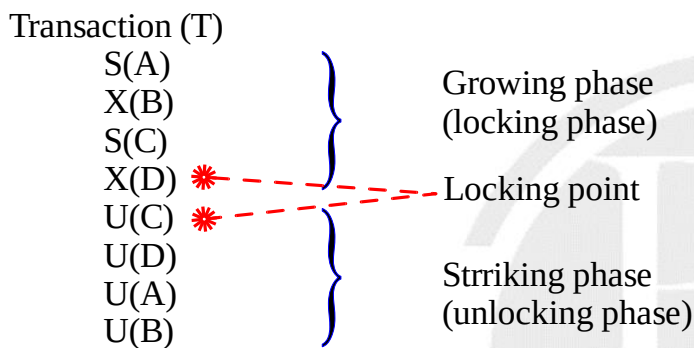
1. Growing Phase

Acquires all the required locks without unlocking any data. Once all locks have been acquired, the transaction is in it locked point.

2. Shrinking phase

Release all locks and can not obtain any new lock governing rules of 2 PL.

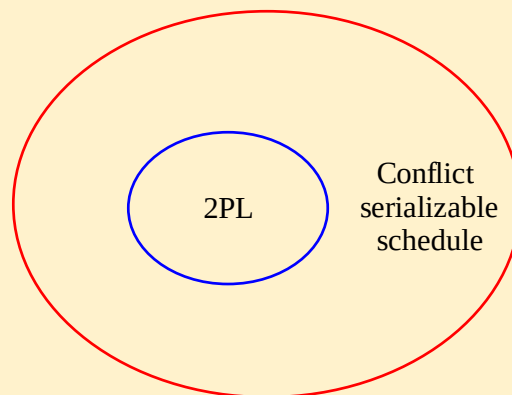
Example:



Important point 1

1. If schedule (s) is executed by 2PL then schedule guaranteed is conflict serializable schedule.
2. Conflict equal schedule is based on lock points order of the transaction.
3. If schedule is not conflict serializable schedule (cyclic precedence graph) then schedule not allowed to execute by 2 PL.

Note: Every schedule which is allowed by 2PL is always conflict serializable, but not every conflict serializable schedule is allowed by 2PL.





5.23.1 Limitations of 2PL

1. 2PL restriction may leads to deadlock.
2. 2PL restriction may leads to starvation.
3. 2PL condition not sufficient to avoid
 - (I) Ir-recoverable schedules
 - (II) Cascading rollback problem
 - (III) lost updated problem

5.24 Strict 2PL protocol

- **Basic 2PL:** Lock request of transaction (T) not allowed in shrinking phase of transaction T
and
Strict recoverable: All exclusive lock of transaction must hold until commit/Rollback of transaction T.

5.24.1 Strict 2PL protocol guarantees

- (a) Serializability.
- (b) Strict recoverable.

Disadvantage:

It is not free form deadlock and starvation.

Example:

Transaction (T)

$\left. \begin{array}{l} S(A) \\ X(B) \\ S(C) \\ X(D) \end{array} \right\} \text{growing phase}$

$\left. \begin{array}{l} U(A) \\ U(B) \\ \text{Commit} \\ U(B) \\ U(D) \end{array} \right\} \text{Strinking phase}$

5.25 Rigorous 2PL

Basic 2 PL : A lock request allowed only in growing phase of transaction and
all locks (shared/Exclusive) of transaction T must be hold until commit or Rollback.

5.26 Conservative 2PL

Transaction (T) must lock all required data itmes in starting phase.

If locks are not available then unlock all data items and re-request all data item lock again.



	Basic 2PL	Strict 2PL	Rigorous 2PL	Conservative 2PL
Guaranteed Serializability	Yes	Yes	Yes	Yes
Guaranteed strict Recoverable	No	Yes	Yes	No
Free from dead lock	No	No	No	Yes
Free from starvation	No	No	No	NO

5.27 Time stamp ordering protocol

Assign global unique time stamp value to each transaction and produces order for transaction submission.

The time stamp values of transaction can be used for transaction identification and to set priority between the transaction.

5.27.1 Time stamp value for each data item

- Assume data item is A
- I. **Read TS value (A):** It is the highest transaction time stamp value that has executed R(A) successfully.
- II. **Write TS value (A):** It is the highest transaction TS value that has executed W(A) successfully.

1. Basic Time stamp ordering protocol:

- If transaction T issues R(A) operation.

If $(WTS(A)) > TS(T)$

then Rollback T

else

{Allow to execute R(A) successfully

set $RTS(A) = \max \{RTS(A), TS(T)\}$

- If transaction T issues W(A) operation

if $(RTS(A)) > TS(T)$

{then Rollback trans (T)}

else if $(WTS(A)) > TS(T)$

{then Roll back T}

Else

{Allow to execute W(A) successfully

set $WTS(A) = \{TS(T)\}$

2. Thomas write rule stamp ordering protocol

- If transaction T issues R(A) operation

if $(WTS(A)) > TS(T)$

{then Rollback transaction T}

else



{allow to execute R(A) successfully

set RTS (A) = max {TS(T), RTS (A)}

- b. If transaction T issue W(A) operation
if (RTS (A) > TS(T))

{then roll back T}

else if (WTS(A)) > TS(T)

{then ignore w(A) of transaction T and continue.}

else

{Allow to execute w(A) successfully

set WTS(A) = TS(T)}

3. Strick time stamp ordering protocol

A transition T_2 that issues a R(A) or W(A) such that $TS(T_2) > WTS(A)$ has it read operation delayed until the transaction T_1 that write the value x has committed or rolled back.

Basic TS ordering protocol	Thomas write TS ordering protocol	Strict TS ordering protocol
1. Ensure conflict serialization	1. Ensure view serialization	1. Ensure conflict serialization
2. Free from deadlock	2. Free from deadlock	2. Free from deadlock
3. Not free from starvation	3. Not free from starvation	3. Not free from starvation
4. Not guaranteed strict recoverable	4. Not guaranteed strict recoverable	4. Guaranteed strict recoverable

